

Model and Architect Your MVP

Start Assignment

- Due Wednesday by 11:59pm
- Points 100
- Submitting a text entry box or a file upload

HW: Model and Architect Your MVP

Advanced Software Engineering — Group Assignment

Overview

Each group turns the persona pain points from the last assignment into a modeled and architected MVP, using every artifact type from the *From Personas to Platforms* lecture. Work in your existing startup group. **Worth 100 points; due October 7, 2026.**

By the end you should be able to:

1. Trace every model element back to a real customer pain point.
1. Draw correct UML: context, use case, sequence, class, activity and state machine diagrams.
1. Keep the models consistent with each other.
1. Choose an architecture and justify its performance and security trade-offs.
1. Describe how one service would be deployed on Kubernetes, using valid manifests.

Use LocalBite from the slides as your reference for notation. Your diagrams must describe **your** MVP.

Step 0: From pain points to MVP scope

Start with a one-page scope that every later artifact must trace back to. No diagram earns credit unless it supports a pain point in this table.

1. List your main personas (2–3) and their top pain points (3–5 in total).
1. For each pain point, write the MVP capability that addresses it and how you will know it worked.
1. Mark what is **out of scope** for the MVP. Say what you are deliberately not building.
1. Give each pain point an ID (PP-1, PP-2 ...). Reuse these IDs in your use cases and your architecture decision record.

Use this table format (LocalBite example row shown):

ID	Persona	Pain point	MVP capability	Use case(s)	Success signal
----	---------	------------	----------------	-------------	----------------

PP-1	Maya, ICU nurse	No time to cook or order after a 12-hour shift	One-tap reorder of a saved meal from a nearby chef	UC-2 Place order	Order placed in under 60 seconds
PP-2	...	...	...	...	...

Deliverables

There are 11 deliverables worth 100 points; modeling (Part A) carries 68 of them.

#	Deliverable	Lecture reference	Points
0	Pain-point scope and traceability table	Personas → models	5
A1	Context diagram	Context models	8
A2	Use case diagram covering all pain points from 0	Use case symbol key + example	10
A3	Textual use cases covering all pain points from 0	Structured use case description	10
A4	Sequence diagrams covering all use cases from A2/A3	Sequence symbol key + example	12
A5	Domain class diagram	Class anatomy, relationships, example	12
A6	State machine diagram	State symbol key + order lifecycle	8
A7	Activity diagram with swimlanes	Activity symbol key + fulfilment example	8
B1	Architecture decision record (ADR)	Patterns, comparison matrix	10

B2	Architecture diagram with trust boundaries	Architecture legend, reference architecture	7
C	Kubernetes manifests for one service	Kubernetes objects, manifest example	10
Total			100

Part A: Modeling artifacts

Every diagram must use correct UML symbols from the lecture's symbol-key slides and carry a one-sentence caption saying which pain points (PP-IDs) it supports.

A1. Context diagram (8 pts)

- Show your MVP as one box inside a dashed system boundary.
- Show every external system and actor it depends on: payments, identity, maps, messaging, data sources, partner APIs.
- Below the diagram, write 2–3 sentences justifying the boundary: one thing you chose to build and one you chose to integrate instead.

A2. Use case diagram (10 pts)

- At least 2 actors, including at least one system actor.
- 6–10 use cases named verb + noun (e.g. "Book appointment"), inside the system boundary.
- At least one «include» and one «extend» relationship, with arrows pointing the right way.
- Number each use case (UC-1, UC-2 ...) and tag it with the PP-ID it addresses.

A3. Textual use cases (10 pts)

- Write the two use cases that matter most to your MVP, in the table format from the lecture.
- Rows: Actors, Description, Data, Stimulus, Response, Pre/post-conditions, Comments.
- Add an **Alternative flows** row with at least one failure path (e.g. payment declined, no availability).

A4. Sequence diagrams (12 pts)

- One sequence diagram for each textual use case in A3.
- Use lifelines named :ClassName, activation bars, and correctly styled messages: synchronous (filled arrowhead), asynchronous (open arrowhead) and return (dashed).
- At least one combined fragment (alt, opt or loop) with guards in brackets.
- Include at least one external system from your context diagram.

A5. Domain class diagram (12 pts)

- 6–12 classes from your problem domain, each with attributes (name: Type) and the key operations.
- Multiplicities on every association.
- At least one generalization and at least one aggregation or composition. Be ready to explain why you chose aggregation or composition.
- Every message in your sequence diagrams must appear as an operation on the receiving class.

A6. State machine diagram (8 pts)

- Choose the core entity of your MVP (e.g. Order, Appointment, Application, Ticket).
- Include an initial pseudostate, at least 5 states and at least one final state.
- Label transitions event [guard] / action, with at least one guard and one action.
- Include at least one failure or cancellation path.

A7. Activity diagram with swimlanes (8 pts)

- Model the end-to-end workflow that relieves your top pain point.
- One swimlane per participant: persona, your system, and any partner or external party.
- Use at least one decision with guards and one fork/join pair for parallel work.

Part B: Architecture, and Part C: Kubernetes manifests

B1. Architecture decision record (10 pts)

One or two pages, in this structure:

1. **Context:** the pain points and non-functional needs that drive the design, e.g. peak load, data sensitivity, team size.
1. **Decision:** your chosen architecture style, e.g. modular monolith, N-tier, microservices, event-driven or serverless.
1. **Alternatives considered:** at least two, and why you rejected each.
1. **Performance consequences:** where latency comes from, what scales and how, and your caching or async choices.
1. **Security consequences:** trust boundaries, authentication and authorization, secrets, and where sensitive data lives.
1. **Cloud-agnostic plan:** which open standard you use for each capability (see the mapping slide), and where you accept lock-in.

B2. Architecture diagram (7 pts)

- Use the lecture's legend: client, service, data store, external system, broker, trust boundary, sync and async arrows.

- Mark every trust boundary. Label each arrow that crosses one with its protocol (e.g. HTTPS, SQL/TLS).
- The components must match your class-diagram clusters and your context diagram's external systems.

1. Kubernetes manifests for one service (10 pts)

- Pick one service from your architecture diagram (B2). No code, Dockerfile or container build is required.
- Write a Deployment for it (2+ replicas, CPU and memory requests and limits, and a securityContext with runAsNonRoot: true) and a Service whose selector matches the Deployment's labels. Use a made-up image name such as yourteam/orders:0.1.0.
- Validate the YAML with kubeconform deploy/*.yaml, which works offline with no cluster. Submit the YAML and a screenshot of the passing output.
- Add one paragraph: how this service scales under load (replicas, a HorizontalPodAutoscaler) and how it is secured (Service type, NetworkPolicy, where secrets come from).

Submission

Submit one shared Git repository per group, plus a PDF compiled from it, by [date].

- **Repository layout:** /docs/scope.md, /docs/models/ (A1–A7), /docs/architecture/ (B1–B2), /deploy/ (Kubernetes YAML and the validation screenshot).
- **Diagram tools:** diagrams-as-code (Mermaid or PlantUML) is preferred. draw.io, Lucidchart or neat hand drawings are also fine. Commit both the source and a PNG or SVG export.
- **PDF:** one document in deliverable order (0, A1–A7, B1–B2, C), each diagram with its PP-ID caption.
- **Contribution statement:** a short table of who led which deliverable. Every member must author at least one diagram.
- **Walkthrough:** each group gets 10 minutes in class to trace one pain point through every artifact, from persona to manifest.

Grading rubric

Each deliverable's points are awarded on the same four criteria. The per-deliverable points are in the Deliverables table.

Criterion	Weight	Excellent (full)	Adequate (about half)	Missing or weak (none)
Notation correctness	30%	Every symbol, arrowhead and label is correct UML	Minor errors, e.g. an «include» arrow reversed or multiplicities missing	Wrong diagram type, or symbols misused throughout

Traceability to pain points	25%	Every element traces to a PP-ID; the scope is clear	Most elements trace; some features have no pain point behind them	Generic features unrelated to the personas
Cross-model consistency	25%	Names, messages, operations, states and actors match across all models	A few mismatches	The models contradict each other
Justification and depth	20%	Trade-offs are explained, with specific performance and security reasoning	Choices stated, but reasoning is thin	No reasoning given

Late submissions lose 10% per day. Deductions for individual deliverables use the same criteria.

Self-check before you submit

Run these checks as a group; they are the consistency rules from the lecture, and graders use them too.

- ☐ Every use case carries a PP-ID, and every pain point has at least one use case.
- ☐ Every actor in the use case diagram appears in the context diagram or is a persona.
- ☐ Each textual use case has a matching sequence diagram with the same actors and steps.
- ☐ Every sequence message is an operation on the receiving class in the class diagram.
- ☐ Every state transition is triggered by an event that appears in a sequence or activity diagram.
- ☐ Every swimlane maps to an actor or to an external system in the context diagram.
- ☐ Class names are identical across all diagrams: no Order in one and Purchase in another.
- ☐ Every trust boundary crossing in B2 has a protocol label and an authentication story in B1.
- ☐ The manifests pass validation, set runAsNonRoot, hold no secrets in plain text, and the Service selector matches the Deployment labels.
- ☐ Every diagram has a caption naming its PP-IDs.